Unveiling Intellectual Property Vulnerabilities of GAN-Based Distributed Machine Learning through Model Extraction Attacks

Mengyao Ma
The University of Queensland
CSIRO's Data61
Brisbane, QLD, Australia
mengyao.ma@uq.edu.au

Shuofeng Liu
The University of Queensland
Brisbane, QLD, Australia
shuofeng.liu@uq.edu.au

M.A.P. Chamikara
CSIRO's Data61
Melbourne, VIC, Australia
chamikara.arachchige@data61.csiro.au

Mohan Baruwal Chhetri
CSIRO's Data61
Melbourne, VIC, Australia
mohan.baruwalchhetri@data61.csiro.au

Guangdong Bai
The University of Queensland
Brisbane, QLD, Australia
g.bai@uq.edu.au

Abstract

Generative Adversarial Networks (GANs), as a cornerstone of artificial intelligence (AI), are widely recognized as the intellectual property (IP) of their owners, given the sensitivity of the training data and the commercial value tied to the models. Model extraction attacks, which aim to steal well-trained proprietary models, pose a significant threat to model IP. Nevertheless, current research predominately focuses on the context of machine learning as a service (MLaaS), where the emphasis lies in understanding the attack knowledge acquired through black-box API queries. This restricted perspective exposes a critical gap in investigating model extraction attacks within realistic distributed settings for generative tasks. In this work, we present the first investigation into model extraction attacks against GANs in distributed settings. We provide a comprehensive attack taxonomy, considering three different levels of knowledge the adversary can obtain in practice. Based on it, we introduce a novel model extraction attack named MoEx, which focuses on the GAN-based distributed learning scenario, i.e., Multi-Discriminator GANs, a typical asymmetric distributed setting. MoEx uses the objective function simulation, leveraging data exchanged during the learning process, to approximate the GAN *generator* owned by the server. We define two attack goals for MoEx, *fidelity extraction* and *accuracy extraction*. Then we comprehensively evaluate the effectiveness of MoEx's two goals with real-world datasets. Our results demonstrate its robust capabilities in extracting generators with high fidelity and accuracy compared with existing methods.

CCS Concepts

• **Security and privacy**; • **Computing methodologies** → **Machine learning**;

Keywords

Distributed learning, Model extraction attack, Generative adversarial network

ACM Reference Format:
Mengyao Ma, Shuofeng Liu, M.A.P. Chamikara, Mohan Baruwal Chhetri, and Guangdong Bai. 2024. Unveiling Intellectual Property Vulnerabilities of GAN-Based Distributed Machine Learning through Model Extraction Attacks. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24), October 21–25, 2024, Boise, ID, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3627673.3679850

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '24, October 21–25, 2024, Boise, ID, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0436-9/24/10
https://doi.org/10.1145/3627673.3679850

1 Introduction

Generative Adversarial Network (GAN) has gained widespread adoption in various fields, such as image generation [18] and image segmentation [6]. As a result, GAN has become a cornerstone in the field of artificial intelligence (AI), earning universal recognition and acclaim for their lightweight nature [17] and groundbreaking performance in generative tasks [8]. The GANs are commonly recognized as the intellectual property (IP) of their owners and demand close protection, given the sensitivity of the training data [21], and the substantial involvement of computing power and human expertise. Moreover, the commercial value intricately tied to GANs and their strategic importance in maintaining a competitive edge adds another layer to the critical need for robust IP protection for these models.

Among various known machine learning attacks [23, 24, 30, 37, 38], *model extraction attacks* [14, 25, 32] are the major threats to the IP of models [34, 35], as they steal a functional copy of a target model. With the stolen model, the adversary can deploy fraudulent services for illicit financial gain, posing significant risks to both commercial and ethical interests [13, 31]. Especially, for GANs, model extraction attack is a severe threat, as for companies that rely on proprietary GAN models for competitive advantage (e.g., in areas like image synthesis, data augmentation, or content generation), a MEA can erode this advantage by enabling competitors to replicate and utilize the same capabilities [13, 31]. Model extraction attacks against deployed models have garnered substantial attention so far, as evidenced by the notable studies against cloud-based ML services such as BigML and Amazon Machine Learning [32].

Existing studies explore the attack capability in extracting exact model properties [26, 33] or approximating model behaviors [14, 25].

They are particularly pronounced in the context of machine learning as a service (MLaaS), where proprietary models become accessible through online interfaces for inference use in a black-box manner. This query-based concentration, however, imposes limitations on the generalization of their attack techniques to complex scenarios, such as generative tasks, introduced by the evolving machine learning landscape. Indeed, due to the explosive demand for diverse training data, the adoption of *distributed training strategy* has become prevalent in many popular large ML models including GANs [28]. In contrast to the MLaaS scenario, dishonest participants in distributed training gain considerably more information, significantly increasing their capability to facilitate advanced extraction attacks. Exploring the full extent of model extraction attacks in the distributed context thus becomes urgent.

Our work. This work presents the first investigation into model extraction attacks against GANs within distributed settings. There are two typical scenarios. One is the *symmetric* situation, where participants possess identical model structures and directly exchange model parameters during communication, rendering the model extraction attacks unnecessary; the other is the *asymmetric* situation, where clients and server have non-identical models, making it susceptible to model extraction attacks aimed at stealing the server's model. Therefore, this work focuses on the asymmetric situation in distributed settings solely. Within this situation, Multi-Discriminators GANs (one type of distributed GANs) is indeed our main target. Distributed GANs can be classified into two categories based on the model structure, i.e., Federated learning GANs (FL-GANs) [9, 29] and Multi-Discriminators GANs(MD-GANs) [10]. In the FLGANs framework, clients must train both the generator and discriminator and then exchange the whole models with the server. This framework is a typical symmetric situation, so we do not consider FLGANs as our attack target. In contrast, the communication of MD-GANs between clients and server only contains generated data and discriminator results. The inherent asymmetric architecture between the server and clients introduces an incentive for dishonest clients to sneak the generator owned by the server. Consequently, we select MD-GANs framework as our attack target.

The challenges MD-GAN poses to model extraction attacks lie in its unique architecture and training process. MD-GAN incorporates multiple *discriminators* distributed across clients while the *generator* is located on the server. Clients and server exchange the generated data and the discriminator results during the training iterations. This architecture renders traditional defenses (e.g., limiting query number [13]) ineffective. The exchanged data provides the adversary with richer information compared to the existing query-based extraction strategy, such as the latent input and the model architecture of the generator.

Under the circumstance we mentioned above, we develop a novel model extraction attack, called MoEx, which aims to leverage the exposed information during the training phase to extract the server generator. We refer to the gained model as the *malicious model*. MoEx simulates an objective function to follow the learning of the target generator. By mimicking the behavior of the target generator in each training round, the malicious model duplicates the target model incrementally until the end of the training.

MoEx targets to achieve two goals, *fidelity extraction* (***Goal #1***) and *accuracy extraction* (***Goal #2***), to perform the model extraction attack. We study the capacity of MoEx's two goals from the attack objectives of *fidelity* and *accuracy*, following the commonly-used performance metrics of GAN [14]. Specifically, we use Fréchet Inception Distance (FID) [1] [12] to measure the similarity within the metrics. Fidelity ensures the similarity in the distribution between data generated by the malicious model and the target model, while accuracy ensures that the malicious model generates data consistent with the distribution of the training data from other benign participants.

We categorize the adversaries of MoEx into three types, distinguishing them based on the levels of knowledge they can obtain in real-world settings, i.e., *network adversary*, *local adversary*, and *full-knowledge adversary*, which are characterized as *black-box*, *grey-box*, and *white-box* models, respectively (see Section 3.2).

We conduct comprehensive analyses, both theoretical and experimental, to evaluate the effectiveness of MoEx. We formally and mathematically derive the distance between the malicious model and the target GAN generator, quantified in terms of the FID, which is commonly used as a similarity function to assess GAN's performance [12]. MoEx is also applied to realistic MD-GANs (AsynDGAN [6] and DCGAN-based MD-GAN [10]) with three prevalent benchmarking datasets (CIFAR100 [19], CelebA [22] and LSUN-Classroom [36]) against our three real-world settings. We first evaluate the effectiveness of the fidelity extraction (***Goal #1***) by achieving remarkably low FID scores of fidelity compared with two existing methods, query-based extraction [13] and two-stage extraction [31] across each setting. Similarly, we evaluate the effectiveness of accuracy extraction (***Goal #2***) by attaining strikingly low FID scores, outperforming the same methods in both grey-box and white-box settings. Furthermore, we compare the fidelity and accuracy between fidelity extraction and accuracy extraction, showing that fidelity extraction maintains higher fidelity, while accuracy extraction ensures higher accuracy. All these demonstrate the robust extraction capabilities of MoEx.

Contributions. This work makes the following contributions.

- **Model extraction in distributed learning context**. We propose the first model extraction attack against GAN-based distributed learning, representing the first step of expanding model extraction attacks from query-based scenarios to realistic distributed learning scenarios.
- **Characterizing model extraction attacks in distributed learning**. Our work is the first that presents a comprehensive taxonomy of model extraction attacks for distributed learning. It defines the fidelity extraction and the accuracy extraction capability of model extraction attacks, and a comprehensive attack taxonomy based on the levels of knowledge the adversary can obtain in real-world settings.
- **Comprehensive evaluation**. Our theoretical and empirical analyses demonstrate the effectiveness of MoEx. The proposed analysis framework can serve as the formal foundation for future research in this area.

2 Preliminaries

In this section, we introduce the general structure of MD-GAN and discuss model extraction attacks.

[1] A lower FID scores demonstrate better effectiveness on fidelity and accuracy.

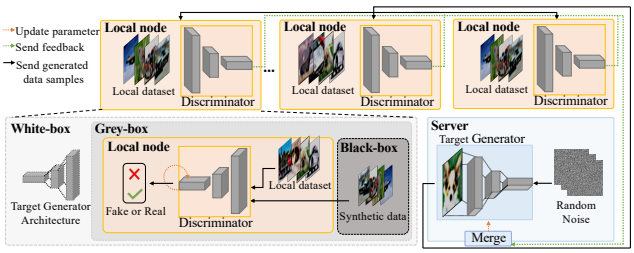


Figure 1: The threat model with three levels of knowledge (white-box, grey-box and black-box settings).

Multi-Discriminator Generative Adversarial Network. The concept of GAN was first proposed by Goodfellow et al. [8]. A GAN consists of a generator G_ω and a discriminator, D_θ , which compete in a two-player min-max game. The generator's goal is to produce data that closely resembles the training data, while the discriminator's task is to distinguish between real data and the generated data. This process helps the generator learn and replicate the distribution of the real training dataset, denoted as P_r, and improve its ability to generate more realistic data. The objective function of GAN is as follows:

$$w^* = \arg\min_{w} \max_{\theta} \left(A_\theta + B_{\theta,w}\right), \tag{1}$$

$$A_\theta = E_{x\sim P_r}\left[\log D_\theta(x)\right], \tag{2}$$

$$B_{\theta,w} = E_{z\sim \mathcal{N}'}\left[\log\left(1 - D_\theta(G_w(z))\right)\right], \tag{3}$$

where $z \sim \mathcal{N}'$ means that each entry of the l-dimensional random vector z follows a normal distribution with fixed parameters. The equation describes the min-max game between the generator G and the discriminator D. D adjusts its parameters θ to maximize A_θ, i.e., the expected good classification on real data and $B_{\theta,\omega}$, the expected good classification on the generated data. G adjusts its parameters ω to minimize $B_{\theta,\omega}$ (ω does not have any impact on A), which means that it tries to minimize the expected good classification of D on the generated data.

MD-GAN [10] employs a distributed architecture that consists of a single generator G located on the server and multiple local discriminators D_k located in K participating parties. During the training phase, each local party k trains its D_k using its own training data X_k. Once trained, the local party k shares the updated parameters with the generator G. Subsequently, G utilizes the aggregated $\{D_k\}$ to update its own parameters, guiding its learning process and improving the generation of synthetic data samples $G(z)$. These samples are dispatched to the K local parties in subsequent iterations. As G incorporates the collective knowledge gathered from each participant, it constitutes the core IP generated by MD-GAN.

Model Extraction of Generative Models. In general, a machine learning model, or typically, a discriminative model [3], is a function that maps input data X to output Y, i.e., $Y = f(X)$. Operating slightly differently from other types of ML models, generative models [8] aim not just to make predictions but also to learn the underlying distribution of input and output data. For example, in tasks such as image generation, the input to the generative model is a latent code, i.e., noise variables z, and the output is a generated instance, such as an image. Discriminative models aim to minimize the loss on the test dataset, while generative models aim to find a set of parameters that allow the generator's distribution to approach that of the training dataset's distribution.

Table 1: Important notations.

Symbol	Explanation
z	random noise input
$G(z)$	target model's output
$G'(z)$	malicious model's output
$\mathcal{S}$	similarity
λ_t, λ_g	eigenvalue of $G(z), G'(z)$
v_t, v_g	eigenvector of $G(z), G'(z)$
m_t, m_g	a value of v_t, v_g
C_t, C_g	Covariance of $G(z), G'(z)$
$\widetilde{m_{t_{*i}}}, \widetilde{m_{g_{*i}}}$	the mean of the ith column of C_t, C_g
M_t, M_g	the mean of v_t, v_g
Tr	the trace of a matrix
$\|\| \cdot \|\|_2^2$	Euclidean distance
$\mathcal{A}_k$	the attacker from local node k
X	the datasets of all the local nodes
X_k	adversary's local dataset
X_T	a set of target generator's output
$X^{(real)}$	set of images considered to be `real` by the discriminator
$X^{(fake)}$	set of images considered to be `fake` by the discriminator
$\hat{X}$	X excludes X_k
PER_D	discriminator's performance in accuracy extraction
ε	discriminator's error on classification

In a model extraction attack against a generative model, the adversary seeks to acquire a replicated model f' that closely mimics the target model f. Existing studies [13, 14] categorize model extraction attacks on GAN generators into two main categories, i.e., *fidelity extraction* and *accuracy extraction*. Fidelity extraction aims to minimize the difference in the distribution between the data generated by the attack model and that by the target generator, while accuracy extraction focuses on minimizing the data distribution between the attack model and the distribution of real data.

3 Threat Model and Problem Definition

In this section, we define two goals of model extraction in the context of distributed GAN, and present a comprehensive attack taxonomy to characterize the adversary's capabilities. Table 1 shows the important notifications we use in our definition and later formalization.

3.1 Attack Goals

Goals. Following the literatures [13, 14], we break down the goals of extracting GANs into *fidelity extraction* and *accuracy extraction* .

- ***Goal #1. Fidelity extraction***. The fidelity extraction aims to learn the output distribution of the target generator, which means reproducing *identical* behaviors of the target generator, including both accuracy and errors of the target model.
- ***Goal #2. Accuracy extraction***. The accuracy extraction aims to learn the training data distribution of the target generator. This goal is particularly crucial in the context of distributed learning, as it enables the adversary to gain distribution of the data of other benign participants, giving the adversary advantages for further privacy attacks, e.g., membership inference attacks [16].

Definitions 1 and 2 present the formal definitions of ***Goal #1*** and ***Goal #2*** respectively.

Definition 1. *(Fidelity extraction). Targeting the generator G, MoEx aims to construct a malicious model G', such that the malicious model's output $G'(z)$ and the target model's output $G(z)$ are similar.*

Definition 2. *(Accuracy extraction). Targeting the generator G, MoEx aims to construct a malicious model G', such that the malicious model's output $G'(z)$ and the subset of the training set $\hat{X}$ are similar.*

Goal #1 and ***Goal #2*** measure the capabilities of MoEx from different perspectives. ***Goal #1*** focuses on reproducing identical behaviors of the target generator, while ***Goal #2*** focuses on extracting the distribution of the training data.

Metrics for two goals. To evaluate the effectiveness of the two attack goals, we introduce *fidelity* and *accuracy* as our evaluation metrics. They both calculate the similarity between two items. In this work, We choose Fréchet Inception Distance (FID) [12] as the similarity function, since it is a widely adopted measurement for image generative models [7, 15].

- ***Fidelity***. Fidelity measures the similarity between $G'(z)$ and $G(z)$, which is denoted as $\mathcal{S}(G'(z), G(z))$.
- ***Accuracy***. Accuracy measures the similarity between $G'(z)$ and $\hat{X}$, which is denoted as $\mathcal{S}(G'(z), \hat{X})$.

3.2 Attack Capabilities

In contrast to the traditional model extraction attacks [14, 25], where the adversary has limited capability of querying the target model, MoEx investigates the threats specific to the distributed settings. In particular, the adversary may gain access to the local data, network communication, or even knowledge about the target generator. Below, we present a comprehensive characterization of the adversary, which is illustrated in Figure 1. Note that without giving favor to the adversary, we assume that MoEx can compromise, at most, only one local node.

- ***Network adversary***. The network adversary, akin to the Dolev-Yao style adversary, can overhear and intercept any message in the network exchanged between a local node and the server. We refer to this type of adversary as the *black-box* adversary, given that they have no access to the local data and knowledge about the generator.
- ***Local adversary***. The local adversary exercises control over one local node. In addition to the information accessible to the network adversary, the local adversary possesses control and visibility into local information, encompassing the local discriminator and local data. We refer to this type of adversary as the *grey-box* adversary.
- ***Full-knowledge adversary***. The full-knowledge adversary is the most powerful adversary that has all the capabilities of the local adversary and additional insight into the architecture of the target generator. As such, we refer to this type of adversary as the *white-box* adversary. In practical terms, the white-box setting represents an insider or a third party possessing knowledge of the deployed model's original architecture.

4 MoEx: Our Approach

In this section, we present MoEx in detail. As shown in Figure 2, MoEx uses objective function simulation during the training phase to generate the malicious model. In fidelity extraction (***Goal #1***) and accuracy extraction (***Goal #2***), the distinction lies in the fact that in accuracy extraction, the adversary can keep the synthetic data sent by the server but filter out categories that existed in their local dataset using the local discriminator.

4.1 Fidelity Extraction

In fidelity extraction, the adversary aims to replicate *both the error and the accuracy* of the target model to the malicious model. To achieve this, the adversary launches attacks to approximate the output distribution of the target generator by leveraging their knowledge. MoEx uses the Mean Squared Error (MSE), as shown in Eq.4,

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{4}$$

to minimize the discrepancy between the output distribution of the malicious model and that of the target server's generator. We give a detailed explanation for MoEx against each setting as follows based on the knowledge that the adversary possesses.

Black-box setting. In the black-box setting, the adversary operates as a network adversary, being not a client of all participants. The adversary can only access the communication process, i.e., the synthetic data generated by the target server's generator that is being transferred from the server to clients. Thus, based on communication information obtained from the distributed network, the adversary can detect the size, channels, and resolution of the synthetic data by *Pillow* [1], a widely used image processing Python library, enabling them to roughly infer the architecture of the target generator. Subsequently, armed with knowledge about the synthetic data, along with the generator's architecture, the adversary can estimate the internal components of the malicious model, such as its layers and blocks. Since the exact architecture of the server's generator is inaccessible, MoEx adopts Convolutional Neural Network (CNN) [2] with appropriate layers and Residual Network (ResNet) [11] with suitable blocks to execute the attack respectively, due to CNN's lightweight architecture and ResNet's strong image generation capability [20, 39, 40].

To closely approximate the output distribution from the target generator, MoEx utilizes the MSE as the simulated loss function for the malicious model as shown in Eq.5,

$$\mathcal{L}_{\mathcal{A}}{}^{Black} = \mathcal{L}(X_A, X_T), \tag{5}$$

where X_A denotes the synthetic data samples generated from the malicious model, and X_T represents the synthetic data samples generated from the target generator. After proper training iterations, a malicious model can be obtained.

Grey-box setting. In the grey-box setting, the adversary acts as a local client to launch the attack, who possesses their own local discriminator and the local dataset. Similar to the black-box setting, the adversary is unaware of the architecture of the server's generator. Therefore, MoEx once again employs CNN and ResNet as model architecture to execute the attack. The difference is in the grey-box setting, it utilizes an advanced simulated loss function

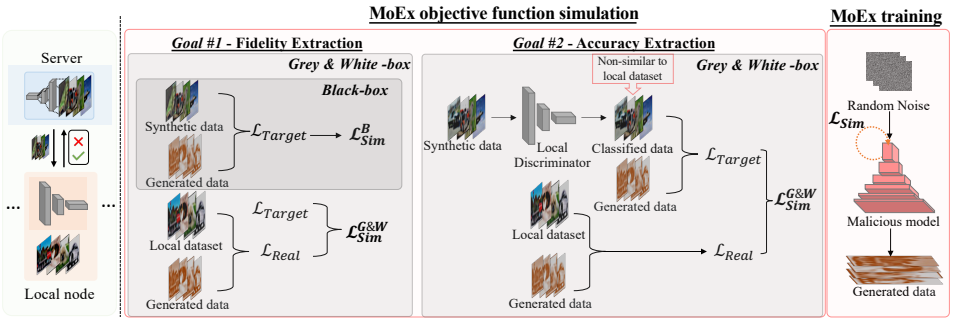


Figure 2: Overview of MoEx's approach.

consisting of two components: the loss from the real local dataset and the loss from the output of the target generator, as shown in Eq.6,

$$\mathcal{L}_{\mathcal{A}}{}^{Grey} = \lambda_{real} * \mathcal{L}(X_A, X_k) + \lambda_{target} * \mathcal{L}(X_A, X_T), \tag{6}$$

where λ_{real} represents the weighted parameter controlling the contribution of the real data owned by the adversary X_k to the loss, and λ_{target} represents the weighted parameter controlling the contribution of the synthetic data from the target generator X_T to the loss. The reason we design Eq.6 is that at the outset of training, the quality of synthetic data samples generated from the target generator is typically poor while in the contrast, the real local data provides more useful information for training, so our aim is to ensure that the proportion of loss attributed to the target generator is relatively small while the loss attributed to the real local dataset is large initially. However, as training progresses and the quality of synthetic data samples improves, we gradually increase the proportion of loss from the target generator compared to that from the real local dataset. To achieve this, MoEx dynamically adjusts the weighted parameters during the training iterations,

$$\begin{aligned} \lambda_{real}(t) &= \lambda_{real}^{init} - \frac{t}{t_{max}} \times (\lambda_{real}^{final} - \lambda_{real}^{init}), \\ \lambda_{target}(t) &= \lambda_{target}^{init} + \frac{t}{t_{max}} \times (\lambda_{target}^{final} - \lambda_{target}^{init}). \end{aligned} \tag{7}$$

where t_{max} represents total duration over training, and λ^{init} and λ^{final} represent the starting and ending value of λ during the training. This approach initiates with a large value for λ_{real} and gradually diminishes it over the course of training steps, while λ_{target} begins with a small value and increases throughout the training iterations. We set the range for λ_{real} and λ_{target} as $(0, 1)$.

White-box setting. In the white-box setting, the adversary not only functions as a local client but also as an insider or third-party of the deployed model. This grants the adversary access to the exact architecture of the server's generator, in addition to the information available to the local adversary. With precise knowledge of the generator, the adversary can train the malicious model using the established architecture, incorporating the advanced simulated loss function outlined in Eq.6.

4.2 Accuracy Extraction

The goal of accuracy extraction is to ensure that the malicious model's outputs are similar to other clients' training data. To achieve this, the adversary has to discriminate the distribution of the generated data by using their local data. This applies only to the scenarios where the adversary is one of the participating clients. Therefore, we consider only grey-box and white-box settings in accuracy extraction.

Grey-box setting. In general, MoEx follows the attack steps outlined in fidelity extraction, with the added step of data classification before training the malicious model using the adversary's local discriminator, as depicted in Figure 2 (***Goal #2***). The classification step allows the adversary to discard synthetic data that is similar to their local data. The rationale behind this is based on the idea that a sample that deviates from the adversary's local data better reflects the distribution of training data from other local clients because if the generator produced an image that the adversary has never seen before, it must have been trained using data from other clients. Additionally, we use the local discriminator to do classification in MoEx instead of a distance-based differentiation method because the former can capture the semantics of the data samples. For instance, two images of the same object with different poses or colors may have a large pixel-level distance, making distance-based methods less effective.

The concrete method involves MoEx utilizing the local discriminator to classify the data samples received from the server, retaining only those samples that are not similar to the adversary's own local data. Since the local discriminator is trained on the adversary's own local data, it is more likely to classify data similar to the adversary's data as `real` and data that is not similar as `fake`. The local discriminator outputs the probabilities of being `real` of local data and synthetic data sent by the server in each batch respectively, so that the difference between the two probabilities can be used to consider whether the synthetic data is similar to the local data or not. MoEx calculates the mean μ of the difference between the two probabilities, and the corresponding squared error σ. We observe that σ is a very small value, rendering the μ a perfect threshold to separate the synthetic data is similar to the local data or not. If the absolute value of the difference exceeds μ, MoEx considers the synthetic data to be dissimilar to the adversary's local data, implying it originates from other clients' training data, which satisfies Eq.8,

$$|\mathcal{P}_{\texttt{real}}[D_{\mathcal{A}}(G(z))] - \mathcal{P}_{\texttt{real}}[D_{\mathcal{A}}(X_k)]| \leq \mu \tag{8}$$

where $\mathcal{P}_{\texttt{real}}[D_{\mathcal{A}}(G(z))]$ and $\mathcal{P}_{\texttt{real}}[D_{\mathcal{A}}(X_k)]$ are the probabilities that the local discriminator outputs `real` for the synthetic data sent from the server and for the adversary's local data samples respectively. In this case, MoEx uses this batch to train the malicious

model; otherwise, MoEx pauses training until the next available batch. The rest training phase of the malicious model remains the same as fidelity extraction.

White-box setting. In the white-box setting, the adversary employs the same approach to conduct the attack as in the grey-box setting we mentioned above. The difference lies in the training of the malicious model, where the adversary can utilize the same model architecture as the server's generator due to their additional knowledge.

5 Formal Analysis

In this section, we mathematically show that our extraction approach can achieve the two attack goals defined in Section 3.1 (Definition 1 and 2). Table 1 presents the notations used in the proof.

Theorem 1 (Fidelity extraction). *Given the noise input z, the similarity between the target generator's output $G(z)$ and the malicious model's output $G'(z)$, denoted as an FID score, is given in Eq.9.*

$$\mathcal{S}(G'(z), G(z)) = FID = ||M_t - M_g||_2^2 + Tr(C_t + C_g - 2(C_tC_g)^{\frac{1}{2}}) \quad (9)$$

where C_t and M_t are $G(z)$ covariance and mean of $G(z)$'s eigenvector, respectively; C_g and M_g are $G'(z)$ covariance and mean of $G'(z)$'s eigenvector, respectively. This similarity and the loss function $\mathcal{L}$ used in the malicious model is relevant, i.e., $\mathcal{S}(G'(z), G(z)) \sim \mathcal{L}$.

To prove Theorem 1, we provide a lemma first introducing the basic properties of eigenvalue and eigenvector.

Lemma 2. *$\Gamma v_i = \lambda_i v_i$, where Γ is a matrix; v_i and λ_i are the eigenvectors and eigenvalues respectively that can represent Γ in its vector space.*

Proof of Theorem 1. According to Lemma 2, We have $G(z)$ and $G'(z)$ satisfy $G(z)v_{t_i} = \lambda_{t_i} v_{t_i}$ and $G'(z)v_{g_i} = \lambda_{g_i} v_{g_i}$, where i is the dimension of $G(z)$ or $G'(z)$.

Now, we break down each term in FID and explain in detail how each of them can be calculated. M_t is the mean of $G(z)$'s eigenvector, so it can be formulated as $M_t = \frac{1}{n}\sum_{i=1}^{n} v_{t_i}$. Similarly, we have $M_g = \frac{1}{n}\sum_{i=1}^{n} v_{g_i}$.

C_t and C_g are the covariance of $G(z)$ and $G'(z)$ respectively, so they can be formulated as Eqs. 10 and 11.

$$C_t = cov([v_{t_1} \quad \dots \quad v_{t_n}]) = cov(\begin{bmatrix} m_{t_{11}} & \dots & m_{t_{1n}} \\ \vdots & \ddots & \vdots \\ m_{t_{n1}} & \dots & m_{t_{nn}} \end{bmatrix}) \quad (10)$$

$$C_g = cov([v_{g_1} \quad \dots \quad v_{g_n}]) = cov(\begin{bmatrix} m_{g_{11}} & \dots & m_{g_{1n}} \\ \vdots & \ddots & \vdots \\ m_{g_{n1}} & \dots & m_{g_{nn}} \end{bmatrix}) \quad (11)$$

According to the computational definition of covariance, we can rewrite Eqs. 10 and 11, then we can obtain Eqs. 12 and 13.

$$C_t = \begin{bmatrix} (m_{t_{*1}} - \widetilde{m_{t_{*1}}}) \times (m_{t_{*1}} - \widetilde{m_{t_{*1}}}) & \dots \\ \vdots & \ddots \end{bmatrix} \quad (12)$$

$$C_g = \begin{bmatrix} (m_{g_{*1}} - \widetilde{m_{g_{*1}}}) \times (m_{g_{*1}} - \widetilde{m_{g_{*1}}}) & \dots \\ \vdots & \ddots \end{bmatrix} \quad (13)$$

The trace of a matrix is the sum of the elements on its main diagonal. As a result, we can mainly focus on the main diagonal without doing any further operations on other elements, making it easy to calculate. We dive into the calculation of the trace part first in FID.

Based on Eqs. 12 and 13, we calculate $C_t + C_g - 2(C_tC_g)^{\frac{1}{2}}$.

$$\begin{aligned} & C_t + C_g - 2(C_tC_g)^{\frac{1}{2}} \\ & = \begin{bmatrix} \frac{1}{n-1}\sum_{i=1}^{n}[(m_{t_{i1}} - \widetilde{m_{t_{*1}}}) - (m_{g_{i1}} - \widetilde{m_{g_{*1}}})]^2 & \dots \\ \vdots & \ddots \end{bmatrix} \end{aligned} \quad (14)$$

As a result, we have

$$\begin{aligned} & Tr(C_t + C_g - 2(C_tC_g)^{\frac{1}{2}}) \\ & = \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{n}[(m_{t_{ij}} - m_{t_{*j}}) - (m_{g_{ij}} - m_{g_{*j}})]^2 \end{aligned} \quad (15)$$

which can be considered as a sum of perfect square trinomial.

Next, we go through $||M_t - M_g||_2^2$ by expanding the equation, and we obtain Eq. 16.

$$\begin{aligned} ||M_t - M_g||_2^2 &= \sqrt{\sum_{i=1}^{n}(v_{t_i} - v_{g_i})^2} \\ &\le \sqrt{(v_{t_1} - v_{g_1})^2} + \dots + \sqrt{(v_{t_n} - v_{g_n})^2} \\ &\le (v_{t_1} - v_{g_1}) + \dots + (v_{t_n} - v_{g_n}) \end{aligned} \quad (16)$$

As a result, the similarity in Eq. 9 satisfies the following equation.

$$\begin{aligned} \text{FID} \le & (v_{t_1} - v_{g_1}) + \dots + \\ & (v_{t_n} - v_{g_n}) + \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{n}[(m_{t_{ij}} - m_{t_{*j}}) - (m_{g_{ij}} - m_{g_{*j}})]^2 \end{aligned} \quad (17)$$

Discussion. The loss function we define in Eq. 4 can be rewritten as another format $\mathcal{L} = \frac{1}{n}[G(z) - G'(z)]^2$, which is equal to $\frac{1}{n}(\lambda_{t_i} - \lambda_{g_i})^2$ in the certain vector space according to Lemma 2. When the value of the loss function reaches its optimal through the gradient-based optimization, the eigenvectors representing the target model and attack model become extremely similar, so that $(v_{t_i} - v_{g_i})$ is a value that can be ignored. Similarly, $m_{t_{ij}} - m_{t_{*j}}$ is close to $m_{g_{ij}} - m_{g_{*j}}$, which means the value of the perfect square trinomial is close to 0. Since both $||M_t - M_g||_2^2$ and $Tr(C_t + C_g - 2(C_tC_g)^{\frac{1}{2}})$ are very small values, the similarity between $G(z)$ and $G'(z)$ is large.

Theorem 3 (Accuracy extraction). *Given the noise input z, the similarity between the malicious model's output $G'(z)$ and other clients datasets $\hat{X}$ is large. This similarity is relevant to loss function $\mathcal{L}$ and discriminator's performance PER_D, i.e., $\mathcal{S}(G'(z), \hat{X}) \sim (\mathcal{L}, PER_D)$.*

Proof of Theorem 3. The proof of Theorem 3 closely parallels the proof of Theorem 1, with the additional consideration of the discriminator's performance PER_D.

Lemma 4. *Theoretically, the output of a GAN can be 100% accurate, i.e., the GAN is able to achieve the optimal to reproduce the images in the training set with 0 error [8].*

We follow the previous work settings shown in Lemma 4 and then explain the correctness of Theorem 3.

Discussion. According to Lemma 4, we are easy to realize that the target model's output $G(z)$ can be equal to the training set X, i.e., $G(z) = X$. Now we assume that node k is the attacker $\mathcal{A}_k$ with

a training dataset X_k. $\mathcal{A}_k$ is capable of eliminating X_k from $G(z)$. Thus, we can have Eq. 18.

$$\hat{X} = X - X_k = G(z) - X_k \tag{18}$$

$\mathcal{A}_k$ leverages the local discriminator to classify whether the images in $G'(z)$ are considered to belong to other clients' datasets by using the difference between two possibilities, shown as Eq. 8. We assume for each iteration, the discriminator's error ε is in a reasonable range, which means X^{real} and X^{fake} can be mostly correctly classified. Thus, we believe Eq. 19 holds.

$$\hat{X} = G(z) - X'_k, \tag{19}$$

where X'_k is the set of data that considered belonging to adversary's local data by MoEx

$\mathcal{A}_k$ uses $\hat{X}$ to train the malicious model using $\mathcal{L}$ in Eq. 4 at this time, and we also denote the output of the malicious model as $G'(z)$. As long as $\mathcal{L}$ reaches its optimal, similarly to Theorem 1, we can derive that $\mathcal{S}(G'(z), \hat{X})$ calculating by FID is small, which means, the similarity between $G'(z)$ and $\hat{X}$ is large. Therefore, $\mathcal{S}(G'(z), \hat{X}) \sim (\mathcal{L}, PER_D)$ holds.

6 Experimental Evaluation

To evaluate the performance of MoEx, we apply it to AsynDGAN and DCGAN-based MD-GAN. We conduct fidelity extraction (***Goal #1***) in black-box, grey-box and white-box settings, and accuracy extraction (***Goal #2***) in grey-box and white-box settings, to provide a comprehensive assessment of its capabilities. The detailed evaluation setups and results are presented as follows.

6.1 Experimental settings

Datasets. In our evaluation, we choose the following benchmark datasets that are widely used in the literature.

- **CIFAR100.** CIFAR100 stands out as a widely used benchmark dataset in machine learning, known for its complexity and robustness. It comprises 60,000 images in 3-channel color, each with dimensions of 32×32 pixels. These images are divided into 100 distinct classes, with each class encompassing 6,000 image samples.
- **CelebA.** CelebFaces Attributes Dataset (CelebA) is a large-scale face attributes dataset with more than 200,000 colored 3-channel celebrity images, each with dimensions of 32×32 pixels. This dataset contains images that display significant variations in pose and background clutter.
- **LSUN-Classroom** LSUN dataset contains millions of color images for scenes, including 10 scene categories and 20 object categories. For our purposes, we focus on the Classroom scene category within the LSUN dataset, which comprises over 150,000 images. Each image is composed of three channels with the dimension of 256×256 pixels.

Learning configuration. We apply MoEx to the DCGAN-based MD-GAN and AsynDCGAN. Following the existing work, we adopt CNN architecture for both generator and discriminator in DCGAN-based MD-GAN. For AsynDCGAN, we adopt ResNet architecture for the generator and CNN architecture for the discriminator.

In the distributed GAN framework, we set up five local nodes (clients) and a server in total. The training data is disjointly allocated among five clients, ensuring that the data for each client is different from that of the other clients. In our black-box setting, the adversary does not belong to any client, while in the grey-box and white-box settings, the adversary acts as one of the clients possessing the training data X_k.

The adversary starts the attack with MoEx from the beginning till the end of the training iterations in all settings. In ***Goal #1*** fidelity extraction, we adopt 5-layer CNN or 6-block ResNet as the malicious model's architecture. In ***Goal #2*** accuracy extraction, we use the same malicious model's architectures as ***Goal #1***, and in addition, we set μ (refer to Section 4.2) to 0.25 for the discriminator's classification task based on our calculation result.

Our evaluation is conducted on a workstation running Ubuntu 22.04.3$LTSx86_64$ as the operating system. The workstation is equipped with two NVIDIA A6000 GPUs.

Baseline. We compare the performance of MoEx with two baselines, query-based extraction and two-stage extraction.

For query-based extraction, we follow the existing work to adopt the DCGAN as the malicious model's architecture. We set the number of queries to 50,000 for collecting the adversary dataset.

For two-stage extraction, we adopt three pre-trained DCGAN models as the malicious models. We set the number of queries to 1000 for collecting the adversary dataset.

Metrics. We use FID-based similarity function, *fidelity* and *accuracy* to evaluate the performance of MoEx on our two goals, fidelity extraction and accuracy extraction. We give the ideal *necessary conditions* for achieving the two goals.

- **Achieving *Goal #1*.** Given

$$\mathcal{S}(G'_{ours}(z), G(z)) \leq \mathcal{S}(G'_{base}(z), G(z)), \tag{20}$$

  such that MoEx achieves ***Goal #1***, i.e., MoEx's effectiveness on fidelity extraction is a success, where $G'_{ours}(z)$ is the final output of our malicious model, and $G'_{base}(z)$ is that of the target generator.
- **Achieving *Goal #2*.** Given

$$\mathcal{S}(G'_{ours}(z), \hat{X}) \leq \mathcal{S}(G'_{base}(z), \hat{X}), \tag{21}$$

  such that MoEx achieves ***Goal #2***, i.e., MoEx's effectiveness on accuracy extraction is a success.

6.2 Performance of MoEx

We evaluate the performance of MoEx in two goals, fidelity extraction and accuracy extraction. Table 2 and Table 3 summarize the performance of MoEx in ***Goal #1*** and ***Goal #2*** across different settings with three datasets respectively. Figure 3 depicts the fidelity score of baseline as the query time increases, as well as that of MoEx on ***Goal #1*** and ***Goal #2*** across different settings as the training epochs progress. Generally speaking, for ***Goal #1***, MoEx performs better than the two baselines in the grey-box and white-box setting, while achieving comparable performance in the black-box setting. For ***Goal #2***, MoEx performs better than the baseline in both grey-box and white-box settings.

6.2.1 Evaluation for **Goal #1**. To evaluate the performance of MoEx on ***Goal #1*** fidelity extraction, we consider three settings mentioned earlier, black-box, grey-box, and white-box settings, based on the adversary's knowledge. Then we use the necessary

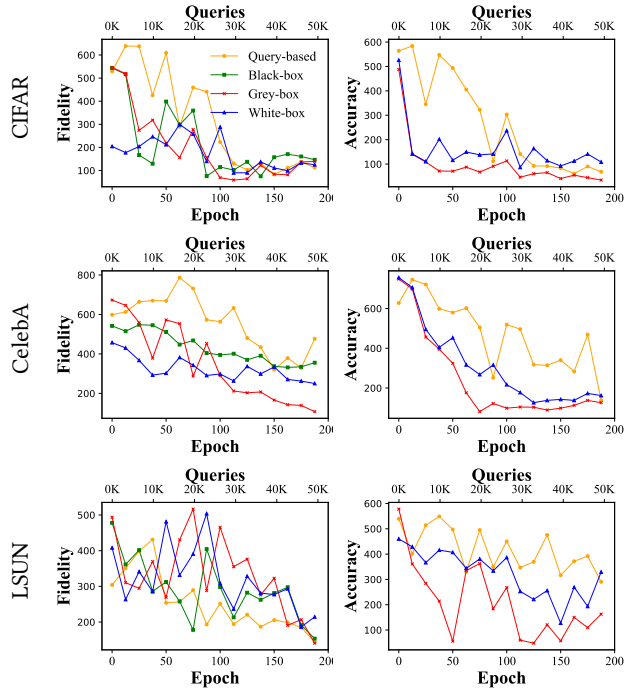
Table 2: Performance of MoEx and baselines on *Goal #1*

	Model structure (Generator architecture)	Query	Two-stage	Blackbox				Greybox				Whitebox	
				Fidelity		Accuracy		Fidelity		Accuracy		Fidelity	Accuracy
				CNN	ResNet	CNN	ResNet	CNN	ResNet	CNN	ResNet		
CIFAR100	D (CNN)	68.48	72.52	68.49	76.24	88.09	93.28	65.09	75.62	81.75	92.84	**65.09**	81.75
	A (ResNet)	99.3	90.87	563.82	105.78	538.95	92.06	569.01	92.23	463.69	83.71	**92.23**	83.71
CelebA	D (CNN)	87.15	80.23	240.06	148.38	241.61	159.27	68.47	99.46	55.36	65.95	**68.47**	65.95
	A (ResNet)	129.1	109.82	459.64	109.11	586.59	87.33	433.48	84.29	540.86	77.92	**84.29**	77.92
LSUN-Classroom	D (CNN)	186.28	166.33	347.44	176.41	332.49	162.54	249.21	**157.21**	252.13	131.24	249.21	252.13
	A (ResNet)	104.86	97.4	490.03	83.23	513.61	83.28	471.22	82.62	532.05	66.26	**82.62**	66.26

"D" stands for DCGAN-based MDGAN structure, "A" stands for AsynDGAN structure. In this table, we use Fréchet Inception Distance (FID) to evaluate the performance. The lower FID score indicates better performance.

Table 3: Performance of MoEx and baseline on *Goal #2*

	Model structure	Query	Greybox				Whitebox	
			Fidelity		Accuracy		Fidelity	Accuracy
			CNN	ResNet	CNN	ResNet		
CIFAR100	D	53.54	95.88	90.29	53.99	**27.49**	95.88	53.99
	A	59.32	520.81	96.64	541.27	66.94	96.64	**66.94**
CelebA	D	81.48	149.62	116.38	54.53	**35.84**	149.62	54.53
	A	89.23	511.75	114.86	418.63	71.14	114.86	**71.14**
LSUN-Classroom	D	71.41	149.97	27.46	145.43	**25.36**	149.97	145.43
	A	67.34	510.20	129.57	472.38	78.46	129.57	**78.46**



(a) Comparison on *Goal #1*. **(b) Comparison on *Goal #2*.**

Figure 3: Comparison of MoEx and baseline across settings

conditions for achieving ***Goal #1*** to justify our assessment. For easy understanding, we highlight in bold the lowest FID scores for attacking different datasets and model architectures in Table 2.

Black-box setting. In the black-box setting, we first evaluate the performance of MoEx in terms of using CNN as the malicious model's architecture. When extracting the structure of AsynDGAN, the fidelity FID scores of MoEx are greater than the two baselines, which means using CNN is not an optimal solution. This can be attributed to the simpler structure of CNN compared to ResNet, leading to relatively inferior image generation quality. When we employ ResNet as the architecture for the malicious model, MoEx receives similar or even lower fidelity FID scores of MoEx across all three datasets compared with two baselines. For instance, our method achieves a fidelity FID score of 109.11 on CelebA, compared to 109.82 of the two-stage extraction baseline, and 83.23 on LSUN-Classroom, compared to 104.86 of the query-based extraction baseline. This satisfies Eq.20, the necessary condition of achieving ***Goal #1***, underscoring the effectiveness of MoEx on fidelity extraction in the black-box setting. The green lines in Figure 3.(a) further illustrate MoEx's performance across three datasets.

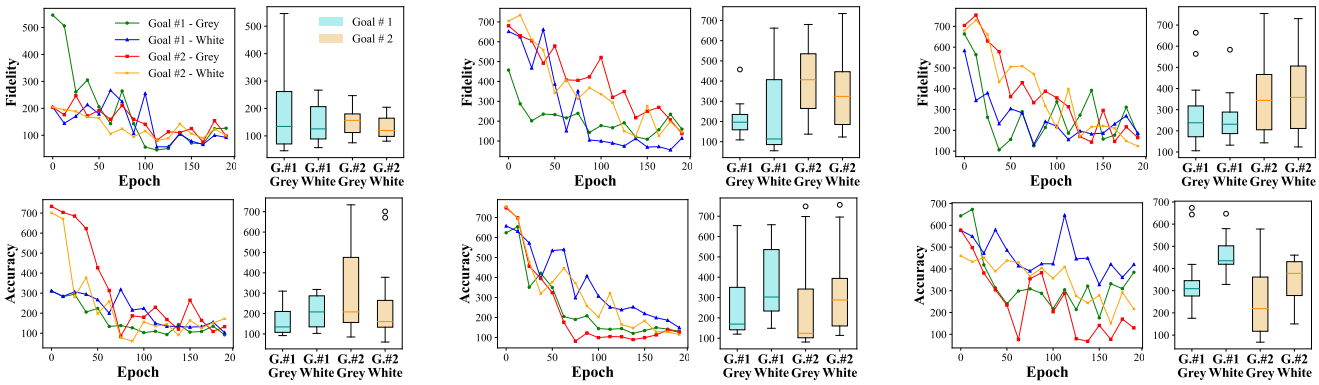
Grey-box setting. In the grey-box setting, using ResNet as the architecture for the malicious model achieves lower fidelity FID scores compared with the two baselines, which means MoEx performs better. For example, MoEx achieves a fidelity FID score of 84.29 on CelebA, compared to 129.1 and 109.82 of the query-based and two-stage extraction baselines, which satisfies Eq. 20. The red lines in Figure 3 depict MoEx's better performance.

White-box setting. Since the adversary knows the architecture of the target generator, MoEx directly uses CNN as the architecture for the malicious model to perform the attack on DCGAN-based MDGAN, and ResNet on AsynDGAN. The result is a subset of that in the grey-box setting, which is mostly better than the two baselines as well, satisfying Eq. 20. The blue lines in Figure 3.(a) show that MoEx performs the best in the white-box setting compared with baselines and other settings.

Takeaways. In ***Goal #1***, based on the results in Table 2, MoEx exhibits comparable performance in the black-box setting, and outperforms two baselines in grey-box and white-box settings. Moreover, according to Figure 3, MoEx performs best in the white-box setting. Additionally, the query-based extraction baselines need to query the target model 50,000 times to get close to MoEx's performance, but MoEx does not have this costly step, presenting another advantage of our method.

6.2.2 Evaluation for **Goal #2**. To evaluate the performance of MoEx on ***Goal #2***, we consider grey-box and white-box settings. We use the necessary conditions for achieving ***Goal #2*** to justify our assessment. The best performance is also highlighted in bold in Table 3.

Grey-box setting. MoEx employs CNN first as the architecture for the malicious model. The results in Table 3 exhibit that MoEx can achieve similar accuracy FID scores to the baseline (e.g., 53.99 *vs* 53.54) on CIFAR10, and outperform the baseline (e.g., 54.53 *vs* 81.48) on CelebA, when executing the attack on DCGAN-based MDGAN. Employing ResNet makes MoEx achieve much lower accuracy



(a) CIFAR (b) CelebA (c) LSUN-Classroom

Figure 4: *Goal 1* vs. *Goal 2* on fidelity and accuracy

FID scores than the baseline (e.g., 35.84 *vs* 81.48) across all three datasets, attacking DCGAN-based MDGAN, and similar results to the baseline (e.g., 71.14 *vs* 89.23) when attacking AsynDGAN. These results satisfy Eq. 21, underscoring the effectiveness of MoEx on accuracy extraction in grey-box setting. The red lines in Figure 3.(b) further illustrate its better performance.

White-box. The results in the white-box setting are a subset of the grey-box setting, which satisfies Eq. 21 as well. The blue lines in Figure 3.(b) also depict MoEx's better performance.

Takeaways. MoEx shows better performance than the baseline in both grey-box and white-box settings in ***Goal #2***. According to Table 3 and Figure 3.(b), we find that, MoEx performs the best in the grey-box setting. The reason is using ResNet to simulate the target model with CNN architecture usually generates better results than using CNN. This gives an implication that the adversary could try ResNet first. Also, for ***Goal #2***, the baseline still needs more than 50,000 queries to approach MoEx's performance, which is much more costly than ours.

6.3 *Goal #1* vs *Goal #2*

To provide a clearer understanding of the objectives of MoEx, we have computed both fidelity FID scores and accuracy FID scores for ***Goal #1*** and ***Goal #2*** across three datasets in grey-box and white-box settings. Figure 4 employs line plots and box plots to visualize the results.

Upon examining the line plot, the fidelity FID scores of ***Goal #1*** are lower than those of ***Goal #2***, indicating that fidelity extraction prioritizes fidelity over accuracy. This suggests that ***Goal #1*** is better suited for replicating the performance of the target generator. Conversely, the accuracy FID scores of ***Goal #2*** outperform those of ***Goal #1***, underscoring the focus of accuracy extraction on accuracy. This suggests that ***Goal #2*** is more effective in capturing the data distribution of the training datasets.

Further examination of the box plot reveals that for fidelity FID scores, the median and minimum values of ***Goal #1*** are consistently lower than those of ***Goal #2***, reinforcing the emphasis of ***Goal #1*** on fidelity. Conversely, for accuracy FID scores, the median and minimum values of ***Goal #2*** are lower than those of ***Goal #1***, highlighting the focus of ***Goal #2*** on accuracy.

7 Related Work

Distributed GANs. Generative Adversarial Networks (GANs) was first proposed by Goodfellow et al. [8]. Distributed GAN frameworks fall into two main categories: Federated Learning GANs (FL-GANs) [9, 29] and MD-GANs [10]. FLGANs involve clients training both generator and discriminator networks, with a server aggregating these networks. FedGAN [29] addresses heterogeneous data sources, ensuring convergence. FeGAN [9] focuses on the system view to overcome model collapse and learning divergence problems arising from a distributed setup.

Model extraction attacks. Model Extraction [14] can be categorized into two main types: fidelity extraction and accuracy extraction, depending on the adversary's objective. Since its inception, several methods have been developed to reduce the number of queries, thereby enhancing attack efficiency. These methods include model extraction using active learning [5, 27] and semi-supervised learning [14]. In fidelity model extraction, the attack model aims to faithfully replicate the predictions of the target model, including any errors. Typical works include model reconstruction from model explanations [25], functionally equivalent extraction [4], and cryptanalytic extraction [14]. Hailong and Jun were among the first to investigate model extraction in generative models [13], proposing defenses against model extraction based on data perturbation.

8 Conclusion

This work pioneers the exploration of model extraction attacks within distributed settings. It contributes a comprehensive attack taxonomy that lays the groundwork for future research. We propose MoEx that consists of the fidelity and accuracy extraction for the GAN-based distributed learning, using objective function simulation to synthesize information exchanged during the learning process. Our theoretical and empirical analyses demonstrate the high extraction capability of proposed strategies.

Acknowledgments

Mengyao Ma is supported by the University of Queensland and CSIRO's Data61 PhD scholarship. This work is partially supported by Australian Research Council Discovery Projects (DP230101196, DP240103068).

References

[1] 2024. Pillow. https://github.com/python-pillow/Pillow.

[2] Saad Albawi, Tareq Abed Mohammed, and Saad Al-Zawi. 2017. Understanding of a convolutional neural network. In *2017 International Conference on Engineering and Technology (ICET)*. Ieee, 1–6.

[3] Yasaman Bahri, Jonathan Kadmon, Jeffrey Pennington, Sam S Schoenholz, Jascha Sohl-Dickstein, and Surya Ganguli. 2020. Statistical mechanics of deep learning. *Annual Review of Condensed Matter Physics* 11 (2020), 501–528.

[4] Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. 2020. Cryptanalytic extraction of neural network models. In *40th Annual International Cryptology Conference (CRYPTO)*. Springer, 189–218.

[5] Varun Chandrasekaran, Kamalika Chaudhuri, Irene Giacomelli, Somesh Jha, and Songbai Yan. 2020. Exploring connections between active learning and model extraction. In *In 29th USENIX Security Symposium*. 1309–1326.

[6] Qi Chang, Hui Qu, Yikai Zhang, Mert Sabuncu, Chao Chen, Tong Zhang, and Dimitris N Metaxas. 2020. Synthetic learning: Learn from distributed asynchronized discriminator gan without sharing medical image data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 13856–13866.

[7] Min Jin Chong and David Forsyth. 2020. Effectively unbiased fid and inception score and where to find them. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 6070–6079.

[8] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. *Advances in Neural Information Processing Systems (NeurIPS)* 27 (2014).

[9] Rachid Guerraoui, Arsany Guirguis, Anne-Marie Kermarrec, and Erwan Le Merrer. 2020. Fegan: Scaling distributed gans. In *Proceedings of the 21st International Middleware Conference (MIDDLEWARE)*. 193–206.

[10] Corentin Hardy, Erwan Le Merrer, and Bruno Sericola. 2019. Md-gan: Multi-discriminator generative adversarial networks for distributed datasets. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 866–877.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778.

[12] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in Neural Information Processing Systems (NeurIPS)* 30 (2017).

[13] Hailong Hu and Jun Pang. 2021. Stealing machine learning models: Attacks and countermeasures for generative adversarial networks. In *Annual Computer Security Applications Conference (ACSAC)*. 1–16.

[14] Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. 2020. High accuracy and high fidelity extraction of neural networks. In *29th USENIX Security Symposium*. 1345–1362.

[15] Haolin Jia, Qifei Wang, Omer Tov, Yang Zhao, Fei Deng, Lu Wang, Chuo-Ling Chang, Tingbo Hou, and Matthias Grundmann. 2023. BlazeStyleGAN: A real-time on-device StyleGAN. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4689–4693.

[16] Jinyuan Jia, Ahmed Salem, Michael Backes, Yang Zhang, and Neil Zhenqiang Gong. 2019. Memguard: Defending against black-box membership inference attacks via adversarial examples. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*.

[17] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. 2021. Alias-Free Generative Adversarial Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[18] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. 2020. Analyzing and Improving the Image Quality of StyleGAN. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[19] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

[20] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. 2021. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE Transactions on Neural Networks and Learning Systems* 33, 12 (2021), 6999–7019.

[21] Shuofeng Liu, Zihan Wang, Minhui Xue, Long Wang, Yuanchao Zhang, and Guangdong. Bai. 2024. Being Transparent is Merely the Beginning: Enforcing Purpose Limitation with Polynomial Approximation. In *33rd USENIX Security Symposium*.

[22] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. 2015. Deep Learning Face Attributes in the Wild. In *Proceedings of International Conference on Computer Vision (ICCV)*.

[23] Mengyao Ma, Yanjun Zhang, Pathum Chamikara Mahawaga Arachchige, Leo Yu Zhang, Mohan Baruwal Chhetri, and Guangdong Bai. 2023. LoDen: Making Every Client in Federated Learning a Defender Against the Poisoning Membership Inference Attacks. In *18th ACM ASIA Conference on Computer and Communications Security (ASIACCS)*. ACM.

[24] Zhongkui Ma, Xinguo Feng, Zihan Wang, Shuofeng Liu, Mengyao Ma, Hao Guan, and Mark Huasong Meng. 2023. Formalizing Robustness Against Character-Level Perturbations for Neural Network Language Models. In *International Conference on Formal Engineering Methods (ICFEM)*. 100–117.

[25] Smitha Milli, Ludwig Schmidt, Anca D Dragan, and Moritz Hardt. 2019. Model reconstruction from model explanations. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FACCT)*. 1–9.

[26] Seong Joon Oh, Bernt Schiele, and Mario Fritz. 2019. Towards reverse-engineering black-box neural networks. *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (2019), 121–144.

[27] Soham Pal, Yash Gupta, Aditya Shukla, Aditya Kanade, Shirish Shevade, and Vinod Ganapathy. 2020. Activethief: Model extraction using active learning and unannotated public data. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 34. 865–872.

[28] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125* 1, 2 (2022), 3.

[29] Mohammad Rasouli, Tao Sun, and Ram Rajagopal. 2020. Fedgan: Federated generative adversarial networks for distributed data. *arXiv preprint arXiv:2006.07228* (2020).

[30] Shaofei Shen, Chenhao Zhang, Alina Bialkowski, Weitong Chen, and Miao Xu. 2024. CaMU: Disentangling Causal Effects in Deep Model Unlearning. In *Proceedings of the 2024 SIAM International Conference on Data Mining (SDM)*. SIAM, 779–787.

[31] Hui Sun, Tianqing Zhu, Wenhan Chang, and Wanlei Zhou. 2024. A two-stage model extraction attack on GANs with a small collected dataset. *Computers & Security* 137 (2024), 103634.

[32] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing Machine Learning Models via Prediction APIs.. In *25th USENIX Security Symposium*, Vol. 16. 601–618.

[33] Binghui Wang and Neil Zhenqiang Gong. 2018. Stealing hyperparameters in machine learning. In *2018 IEEE symposium on security and privacy (S&P)*. IEEE, 36–52.

[34] Shuang Wang, Jiawen Wu, Zian Yuan, An Gao, and Weitong Tony Chen. 2024. Makespan minimization for workflows with multiple privacy levels. *Future Generation Computer Systems* 159 (2024), 39–50.

[35] Zihan Wang, Zhongkui Ma, Xinguo Feng, Ruoxi Sun, Hu Wang, Minhui Xue, and Guangdong. Bai. 2024. CoreLocker: Neuron-level Usage Control. In *IEEE Symposium on Security and Privacy (S&P)*. 222–222. https://doi.org/10.1109/SP54263.2024.00182

[36] Fisher Yu, Yinda Zhang, Shuran Song, Ari Seff, and Jianxiong Xiao. 2015. LSUN: Construction of a Large-scale Image Dataset using Deep Learning with Humans in the Loop. *arXiv preprint arXiv:1506.03365* (2015).

[37] Chenhao Zhang, Shaofei Shen, Yawen Zhao, Weitong Tony Chen, and Miao Xu. 2024. GENIU: A Restricted Data Access Unlearning for Imbalanced Data. *arXiv preprint arXiv:2406.07885* (2024).

[38] Yanjun Zhang, Guangdong Bai, Pathum Chamikara Mahawaga Arachchige, Mengyao Ma, Liyue Shen, Jingwei Wang, Surya Nepal, Minhui Xue, Long Wang, and Joseph Liu. 2023. AgrEvader: Poisoning Membership Inference Against Byzantine-robust Federated Learning. In *The ACM Web Conference (WWW)*.

[39] Yan Zhou, Shaochang Chen, Yiming Wang, and Wenming Huan. 2020. Review of research on lightweight convolutional neural networks. In *2020 IEEE 5th Information Technology and Mechatronics Engineering Conference (ITOEC)*. IEEE, 1713–1720.

[40] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A Efros. 2017. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2223–2232.